

```
from google.colab import drive
import os

drive.mount('/content/drive')

os.makedirs('/content/drive/MyDrive/India_Runs_Hackathon', exist_ok=True)

print("Success! Your drive is mounted and the folder is ready.")
```

```
Mounted at /content/drive
Success! Your drive is mounted and the folder is ready.
```

```
!pip install -q sentence-transformers pandas numpy scikit-learn
print("AI dependencies successfully installed!")
```

```
AI dependencies successfully installed!
```

```
import pandas as pd
import numpy as np
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

print("Loading semantic search AI (takes 10-15 seconds)...")
ai_model = SentenceTransformer('all-MiniLM-L6-v2')

job_requirement = "Looking for a Data Scientist expert in Python, buildi

names = ['Rahul Sharma', 'Amit Patel', 'Sneha Kulkarni', 'Vikram Singh']
exp = [5, 1, 4, 8]
resumes = ["Backend software engineer focused on Java, Spring Boot, and

df = pd.DataFrame({'name': names, 'experience_years': exp, 'resume_text'

print("Analyzing candidates...")
job_vector = ai_model.encode([job_requirement])
candidate_vectors = ai_model.encode(df['resume_text'].tolist())

scores = cosine_similarity(job_vector, candidate_vectors).flatten()
df['match_score'] = scores

ranked_df = df.sort_values(by='match_score', ascending=False).reset_inde
ranked_df['final_rank'] = ranked_df.index + 1

print("\n🏆 --- AI CANDIDATE RANKINGS --- 🏆")
print(ranked_df[['final_rank', 'name', 'match_score']])
```

```
Loading semantic search AI (takes 10-15 seconds)...
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:11
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your setting
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to accept
warnings.warn(
```

```
modules.json: 100% 349/349 [00:00<00:00, 9.82kB/s]
config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 4.52kB/s]
README.md: 100% 10.5k/10.5k [00:00<00:00, 407kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please s
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticat
sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 5.38kB/s]
config.json: 100% 612/612 [00:00<00:00, 51.6kB/s]
model.safetensors: 100% 90.9M/90.9M [00:00<00:00, 143MB/s]
Loading weights: 100% 103/103 [00:00<00:00, 385.03it/s, Materializing param=pooler.dense.weight]
```

```
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
```

```
Key | Status | |
-----+-----+---
embeddings.position_ids | UNEXPECTED | |
```

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture

```
tokenizer_config.json: 100% 350/350 [00:00<00:00, 14.0kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 8.08MB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 15.7MB/s]
special_tokens_map.json: 100% 112/112 [00:00<00:00, 8.01kB/s]
config.json: 100% 190/190 [00:00<00:00, 17.5kB/s]
```

Analyzing candidates...

```
🏆 --- AI CANDIDATE RANKINGS --- 🏆
final_rank  name  match_score
0           1  Sneha Kulkarni  0.579198
1           2    Amit Patel  0.504889
2           3   Rahul Sharma  0.292739
3           4  Vikram Singh  0.182337
```

```
!unzip /content/drive/MyDrive/India_Runs_Hackathon/*.zip -d /content/dri
```

```
Archive: /content/drive/MyDrive/India_Runs_Hackathon/[PUB] India_runs_da
creating: /content/drive/MyDrive/India_Runs_Hackathon/[PUB] India_runs
creating: /content/drive/MyDrive/India_Runs_Hackathon/[PUB] India_runs
```

```
inflating: /content/drive/MyDrive/India_Runs_Hackathon/[PUB] India_runs
inflating: /content/drive/MyDrive/India_Runs_Hackathon/[PUB] India_runs
inflating: /content/drive/MyDrive/India_Runs_Hackathon/[PUB] India_runs
inflating: /content/drive/MyDrive/India_Runs_Hackathon/[PUB] India_runs
inflating: /content/drive/MyDrive/India_Runs_Hackathon/[PUB] India_runs
inflating: /content/drive/MyDrive/India_Runs_Hackathon/[PUB] India_runs
inflating: /content/drive/MyDrive/India_Runs_Hackathon/[PUB] India_runs
inflating: /content/drive/MyDrive/India_Runs_Hackathon/[PUB] India_runs
```

```
!ls /content/drive/MyDrive/India_Runs_Hackathon/
```

```
'[PUB] India_runs_data_and_ai_challenge'
'[PUB] India_runs_data_and_ai_challenge.zip'
```

```
!ls "/content/drive/MyDrive/India_Runs_Hackathon/[PUB] India_runs_data_a
```

```
India_runs_data_and_ai_challenge
```

```
!ls "/content/drive/MyDrive/India_Runs_Hackathon/[PUB] India_runs_data_a
```

```
candidate_schema.json  README.docx          sample_submission.csv
candidates.jsonl        redrob_signals_doc.docx  submission_spec.docx
job_description.docx    sample_candidates.json
```

```
import pandas as pd
path = "/content/drive/MyDrive/India_Runs_Hackathon/[PUB] India_runs_dat
print("Path shortcut set successfully!")
```

```
Path shortcut set successfully!
```

```
submission_df = pd.read_csv(path + "sample_submission.csv")
print("--- EXACT SUBMISSION COLUMNS REQUIRED ---")
print(submission_df.columns.tolist())
print("\n--- PREVIEW OF SUBMISSION FORMAT ---")
print(submission_df.head(3))
```

```
--- EXACT SUBMISSION COLUMNS REQUIRED ---
['candidate_id', 'rank', 'score', 'reasoning']
```

```
--- PREVIEW OF SUBMISSION FORMAT ---
  candidate_id  rank  score \
0  CAND_0004989    1  0.992
1  CAND_0001195    2  0.984
2  CAND_0003114    3  0.976
```

```

                                reasoning
0  HR Manager with 6.1 yrs; 9 AI core skills; res...
1  HR Manager with 8.7 yrs; 9 AI core skills; res...
2  ML Engineer with 6.4 yrs; 4 AI core skills; re...

```

```

candidates_df = pd.read_json(path + "candidates.jsonl", lines=True)
print("--- REAL CANDIDATE DATA COLUMNS ---")
print(candidates_df.columns.tolist())
print("\n--- FIRST CANDIDATE PREVIEW ---")
print(candidates_df.iloc[0])

```

```

--- REAL CANDIDATE DATA COLUMNS ---
['candidate_id', 'profile', 'career_history', 'education', 'skills', 'cer

--- FIRST CANDIDATE PREVIEW ---
candidate_id                                CAND_00000001
profile                                     {'anonymized_name': 'Ira Vora', 'headline': 'B...
career_history                             [{'company': 'Mindtree', 'title': 'Backend Eng...
education                                  [{'institution': 'Lovely Professional Universi...
skills                                     [{'name': 'Tailwind', 'proficiency': 'intermed...
certifications                             []
languages                                  [{'language': 'English', 'proficiency': 'profe...
redrob_signals                             {'profile_completeness_score': 86.9, 'signup_d...
Name: 0, dtype: object

```

```

!pip install -q docx2txt
print("Word document parser loaded!")

```

Word document parser loaded!

```

import docx2txt
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity

print("Step 1: Extracting target requirements from job")
job_desc_text = docx2txt.process(path + "job_descripti

print("Step 2: Processing complex nested data patterns")
# Parse nested fields into clean text streams without
candidates_df['headline'] = candidates_df['profile'].a
candidates_df['skills_text'] = candidates_df['skills']

# Combine all profile attributes into a single searcha
candidates_df['search_profile'] = "Role Focus: " + can

print("Step 3: Generating mathematical match vectors v")
job_vector = ai_model.encode([job_desc_text])
candidate_vectors = ai_model.encode(candidates_df['sea

```

```

print("Step 4: Scoring and prioritizing candidates..."
raw_scores = cosine_similarity(job_vector, candidate_v
candidates_df['score'] = raw_scores

# Sort immediately by score to find the best talent ma
final_submission = candidates_df.sort_values(by='score
final_submission['rank'] = final_submission.index + 1

print("Step 5: Synthesizing the custom 'reasoning' des
# Dynamically create the explanation text column using
final_submission['reasoning'] = "Candidate background

print("Step 6: Cleaning columns to match requested sch
# Select ONLY the 4 exact columns requested in sample_
submission_columns = ['candidate_id', 'rank', 'score',
final_output_table = final_submission[submission_colum

print("Step 7: Writing your finalized submission file
final_output_table.to_csv(path + "my_final_submission.

print("\n🎉 CONGRATULATIONS! Pipeline Complete. Preview
print(final_output_table.head(5))

```

Step 1: Extracting target requirements from job_description.docx...

Step 2: Processing complex nested data patterns from candidates.jsonl...

Step 3: Generating mathematical match vectors via AI brain...

Batches: 100%

3125/3125 [38:41<00:00, 2.29it/s]

Step 4: Scoring and prioritizing candidates...

Step 5: Synthesizing the custom 'reasoning' descriptions required by judg

Step 6: Cleaning columns to match requested schema specification...

Step 7: Writing your finalized submission file straight to Google Drive..

🎉 CONGRATULATIONS! Pipeline Complete. Previewing top candidates:

	candidate_id	rank	score	\
0	CAND_0091454	1	0.626685	
1	CAND_0092796	2	0.623638	
2	CAND_0004628	3	0.616589	
3	CAND_0030732	4	0.611797	
4	CAND_0043829	5	0.609690	

	reasoning
0	Candidate background emphasizes Mechanical Eng...
1	Candidate background emphasizes Mechanical Eng...
2	Candidate background emphasizes AI Research En...
3	Candidate background emphasizes Civil Engineer...
4	Candidate background emphasizes AI Research En...

